

Vorlesung 3: Zahldarstellung & Rundungsfehler

§2.2 Gleitkommazahlen | §2.3 IEEE 754 | §2.4 Runden & Maschinenoperationen | §2.5 Auslöschung

1 Darstellung reeller Zahlen: Zahlssysteme

Satz 2.11 — Darstellung im Stellenwertsystem

Jede reelle, von Null verschiedene Zahl $x \in \mathbb{R}$ lässt sich darstellen als

$$x = \pm \beta^e \cdot \sum_{i=1}^{\infty} d_i \beta^{-i},$$

wobei $\beta \in \mathbb{N}$, $\beta \geq 2$ die **Basis** ist, $d_i \in \{0, \dots, \beta - 1\}$ die **Ziffern** sind, $e \in \mathbb{Z}$ der **Exponent** und $d_1 \neq 0$ (normierte Darstellung).

Beispiel — Darstellung von 6.5 in verschiedenen Systemen

System	Basis β	Ziffern	6.5 dargestellt als
Dezimal	10	$0, 1, \dots, 9$	6.5_{10}
Binär	2	$0, 1$	$110.1_2 \quad (= 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1})$
Oktal	8	$0, 1, \dots, 7$	$6.4_8 \quad (= 6 \cdot 8^0 + 4 \cdot 8^{-1})$
Hexadezimal	16	$0, \dots, 9, A, \dots, F$	$6.8_{16} \quad (= 6 \cdot 16^0 + 8 \cdot 16^{-1})$

2 Gleitkommazahlen

Definition 2.13 — Gleitkommazahlen $\mathbb{F}(\beta, t, e_{\min}, e_{\max})$

Sei $\beta \in \mathbb{N}$, $\beta \geq 2$ (Basis), $t \in \mathbb{N}$ (Mantissenlänge), $e_{\min} < 0 < e_{\max}$, $e_{\min}, e_{\max} \in \mathbb{Z}$ (Exponentenschranken). Die Menge der **Gleitkommazahlen** ist:

$$\mathbb{F}(\beta, t, e_{\min}, e_{\max}) := \left\{ \pm \beta^e \sum_{i=1}^t d_i \beta^{-i} \mid d_i \in \{0, \dots, \beta - 1\}, d_1 \neq 0, e \in [e_{\min}, e_{\max}] \right\} \cup \{0\}.$$

Interpretation:

- Die **Mantisse** $m = \sum_{i=1}^t d_i \beta^{-i} \in [1/\beta, 1)$ speichert die signifikanten Ziffern.
- Der **Exponent** e verschiebt das Komma.
- $d_1 \neq 0$ erzwingt Normiertheit (eindeutige Darstellung).
- Auf dem Rechner sind **nur endlich viele** Zahlen speicherbar: $|\mathbb{F}| < \infty$.

Lemma 2.16 — Kleinste und größte Gleitkommazahl

In $\mathbb{F} = \mathbb{F}(\beta, t, e_{\min}, e_{\max})$ gilt:

$$x_{\min} := \min\{x \in \mathbb{F} \mid x > 0\} = \beta^{e_{\min}-1},$$

$$x_{\max} := \max\{x \in \mathbb{F}\} = \beta^{e_{\max}} \cdot (1 - \beta^{-t}).$$

Beweis: Kleinste positive Zahl: $d_1 = 1$, alle anderen $d_i = 0$, $e = e_{\min} \Rightarrow x_{\min} = \beta^{e_{\min}} \cdot \beta^{-1} = \beta^{e_{\min}-1}$.
Größte Zahl: alle $d_i = \beta - 1$, $e = e_{\max} \Rightarrow x_{\max} = \beta^{e_{\max}} \cdot \sum_{i=1}^t (\beta - 1)\beta^{-i} = \beta^{e_{\max}}(1 - \beta^{-t})$.

Lemma 2.17 — Gleichmäßige relative Genauigkeit

Der Abstand zwischen einer Gleitkommazahl $x \neq 0$ und der nächstgelegenen Gleitkommazahl ist:

- mindestens $\frac{1}{2} \varepsilon_M |x|$,
- höchstens $\varepsilon_M |x|$,

wobei $\varepsilon_M := \beta^{1-t}$ der Abstand von 1 zur nächstgrößeren Gleitkommazahl (**Maschinenepsilon**) ist.
 $\Rightarrow$ Gleitkommazahlen haben **gleichmäßige relative Genauigkeit** (nicht absolute). Die Dichte der Zahlen in $\mathbb{F}$ nimmt mit wachsendem $|x|$ ab — sie sind auf dem Zahlenstrahl **nicht gleichmäßig verteilt**.

Denormalisierte Zahlen

Denormalisierte Zahlen erlauben $d_1 = 0$ bei $e = e_{\min}$. Sie schließen die Lücke zwischen 0 und $x_{\min}$.

Nachteil: Lemma 2.17 gilt hier **nicht** — in diesem Bereich ist die relative Genauigkeit reduziert ("gradual underflow").

3 IEEE 754 Standard

IEEE 754 — Formate im Überblick

Typ	Bits	Vorzeichen	Exponent	Mantisse	$x_{\min}$	$x_{\max}$
Single (float32)	32	1	8	23+1	$\sim 10^{-38}$	$\sim 10^{+38}$
Double (float64)	64	1	11	52+1	$\sim 10^{-308}$	$\sim 10^{+308}$

Implizites erstes Bit: Da Basis $\beta = 2$ und $d_1 \neq 0 \Rightarrow d_1 = 1$ immer. Dieses Bit wird **nicht gespeichert** $\Rightarrow$ effektiv 23 + 1 bzw. 52 + 1 Mantissenbits.

Spezialfälle in IEEE 754:

- `inf`: Überlauf (z. B. `3*np.inf`)
- `nan` (*Not a Number*): undefinierte Operation (z. B. `np.inf - np.inf`)

Python: IEEE-754-Grenzen abfragen

```
import numpy as np
```

```
print(np.finfo(float).tiny)    # 2.225e-308 (xmin, double)
print(np.finfo(float).max)     # 1.798e+308 (xmax, double)
print(np.finfo(float).eps)     # 2.220e-16 (Maschinenepsilon, double)
print(np.finfo(np.float32).eps) # 1.192e-07 (Maschinenepsilon, single)
```

```
print(3 * np.inf)          # inf
print(np.inf - np.inf)    # nan
```

Rundungseinheit: $u_{\text{double}} \approx 1.11 \cdot 10^{-16}$, $u_{\text{single}} \approx 5.96 \cdot 10^{-8}$.

Niedrigpräzisionsarithmetik (ML)

Tiefe neuronale Netze sind robust gegen Ungenauigkeiten $\Rightarrow$ reduzierte Formate:

Format	Bits	Exponent	Mantisse
BF16	16	8 (wie Float32)	7+1
FP16	16	5	10+1
INT8	8	–	8 (Quantisierung, 256 Werte)
INT4	4	–	4 (Quantisierung, 16 Werte)

Quantisierung: Gewichte W eines neuronalen Netzes werden auf ein diskretes Gitter abgebildet. Vorteil: geringerer Speicher- und Rechenaufwand. Nachteil: reduzierte Darstellungsgenauigkeit.

4 Runden und Maschinenoperationen

Rundungsabbildung und Rundungseinheit

Sei $\text{Bereich}(\mathbb{F}) := \{x \in \mathbb{R} \mid x_{\min} \leq |x| \leq x_{\max}\} \cup \{0\}$.

Die **Rundungsabbildung** $\text{rd} : \text{Bereich}(\mathbb{F}) \rightarrow \mathbb{F}$ ordnet jeder reellen Zahl x die nächstgelegene Gleitkommazahl zu.

Bei Gleichabstand (exakt zwischen zwei GKZ) rundet IEEE 754 zur **geraden** letzten Mantissenziffer (*round half to even*). Dadurch heben sich Rundungsfehler statistisch auf.

Die **Rundungseinheit** (unit roundoff) ist:

$$u = u(\mathbb{F}) := \frac{1}{2} \beta^{1-t}.$$

Satz 2.20 — Relativer Rundungsfehler

Für alle $x \in \text{Bereich}(\mathbb{F})$ existiert ein $\delta \in \mathbb{R}$ mit

$$\text{rd}(x) = x(1 + \delta), \quad |\delta| \leq u.$$

Beweisidee: O.B.d.A. $x > 0$. Sei e der zugehörige Exponent, $m = x/\beta^e \in [1/\beta, 1)$. Runden der Mantisse auf t Stellen ergibt Fehler $\leq \frac{1}{2}\beta^{-t}$. Relativiert auf $x = \beta^e m$ ergibt sich $|\delta| \leq \frac{1}{2}\beta^{1-t} = u$.

Satz 2.22 — Alternative Darstellung

Für alle $x \in \text{Bereich}(\mathbb{F})$ existiert ein $\delta \in \mathbb{R}$ mit

$$\text{rd}(x) = \frac{x}{1 + \delta}, \quad |\delta| \leq u.$$

(Beweis: Übungsaufgabe, nutze Satz 2.20.)

4.1 Maschinenoperationen

Definition — Maschinenoperationen $\oplus, \ominus, \otimes, \oslash$

Die exakten Grundoperationen $+, -, \cdot, /$ werden ersetzt durch Maschinenoperationen, die das Ergebnis runden:

$$x \oplus y = \text{rd}(x + y) = (x + y)(1 + \delta), \quad |\delta| \leq u.$$

Analog für $\ominus, \otimes, \oslash$.

Wichtig: Maschinenoperationen erfüllen Assoziativ- und Distributivgesetz nur näherungsweise, **nicht exakt**. D. h. $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$ möglich!

Beispiel 2.24 — Zwei Algorithmen für $y = x_1^2 - x_2^2$

$y = x_1^2 - x_2^2 = (x_1 + x_2)(x_1 - x_2)$. Zwei Algorithmen:

Algorithmus A: $\hat{y}_A = (x_1 \otimes x_1) \ominus (x_2 \otimes x_2)$

Algorithmus B: $\hat{y}_B = (x_1 \oplus x_2) \otimes (x_1 \ominus x_2)$

Beide exakt äquivalent — aber: Falls $x_1 \approx x_2$, leidet **Algorithmus A** unter Auslöschung ($x_1^2 \approx x_2^2$, Differenz fast gleicher Zahlen). Algorithmus B ist in diesem Fall stabiler.

5 Katastrophale Fehlerfortpflanzung

Beispiel 2.25 — Rekursion $I_k = e - k \cdot I_{k-1}$

Sei $I_k = \int_0^1 x^k \exp(x) dx$. Es gilt die Rekursion:

$$I_0 = e - 1, \quad I_k = e - k \cdot I_{k-1}, \quad k \geq 1.$$

Problem: Jeder Schritt multipliziert den aktuellen Fehler ΔI_{k-1} mit k :

$$\Delta I_k \approx k \cdot \Delta I_{k-1} \quad \Rightarrow \quad \Delta I_k \approx k! \cdot \Delta I_0.$$

Ein anfänglicher Fehler $\Delta I_0 \sim 10^{-15}$ wächst auf $\Delta I_{25} \sim 10^8$ an — **katastrophale Fehlerfortpflanzung**.

Beobachtung aus der Tabelle (Vorlesung): Für $k \leq 12$ noch vertretbarer Fehler ($\sim 10^{-8}$), ab $k \approx 18$ völlig falsche (sogar negative) Werte.

Achtung

Ein einfacher, mathematisch korrekter Algorithmus kann **massiv instabil** sein. Die Rekursion ist gut konditioniert (das Problem selbst ist stabil), aber der Algorithmus verstärkt Rundungsfehler exponentiell.

Abhilfe: Rückwärtsrekursion (stabile Alternative — Thema späterer Vorlesungen).

6 Numerische Auslöschung (§2.5)

Definition — Numerische Auslöschung

Numerische Auslöschung tritt auf beim Subtrahieren zweier fast gleicher, bereits rundungsfehlerbehafteter Zahlen. Sie ist die **Hauptursache** massiver Rundungsfehlerverstärkung.

Seien $\tilde{a} = \text{rd}(a) = a(1 + \delta_a)$, $\tilde{b} = \text{rd}(b) = b(1 + \delta_b)$ mit $|\delta_a|, |\delta_b| \leq u$.

Für $a \approx b$ gilt $a - b \approx 0$, aber der absolute Fehler in $\tilde{a} - \tilde{b}$ ist unverändert $\sim u|a|$. Der **relative Fehler** in der Differenz:

$$\frac{|\tilde{a} - \tilde{b}| - |a - b|}{|a - b|} \sim \frac{u|a|}{|a - b|} \gg 1 \quad \text{wenn } a \approx b.$$

Beispiel: $f(x) = (1 - \cos x)/x^2$ für kleines x

Sei $x = 1.2 \times 10^{-5}$.

- $\cos(x)$ auf 10 Dezimalstellen: $c = 0.9999\,9999\,99$
- $1 - c = 0.0000\,0000\,01$ (Auslöschung! Alle signifikanten Stellen verloren)
- $(1 - c)/x^2 = 0.6944\dots$ (völlig falsch; korrekter Wert ≈ 0.5)

Korrekte Berechnung: trigonometrische Identität nutzen:

$$f(x) = \frac{1 - \cos x}{x^2} = \frac{2 \sin^2(x/2)}{x^2}.$$

Kein Auslöschungsproblem, da $\sin(x/2) \approx x/2$ für kleines x nicht fast verschwindet.

7 Zusammenfassung: Formalisierung

Begriff	Definition	Typischer Wert
Maschinenepsilon ε_M	Abstand von 1 zur nächsten GKZ: β^{1-t}	double: 2.22×10^{-16}
Rundungseinheit u	$u = \varepsilon_M/2 = \frac{1}{2}\beta^{1-t}$	double: 1.11×10^{-16}
Rel. Rundungsfehler	$ \delta \leq u$ mit $\text{rd}(x) = x(1 + \delta)$	Immer $\leq u$
Auslöschung	Subtraktion fast gleicher Zahlen	Rel. Fehler $\gg u$ möglich
Instabilität	Fehlerverstärkung $\gg$ Kondition	Faktor $k!$ bei I_k -Rekursion

Prüfungscheckliste — Vorlesung 3

- ☐ Darstellung einer reellen Zahl im Stellenwertsystem $x = \pm\beta^e \sum d_i \beta^{-i}$ erklären; 6.5 in Binär, Oktal, Hex umrechnen. → *Satz 2.11*

- ☐ Menge der Gleitkommazahlen $\mathbb{F}(\beta, t, e_{\min}, e_{\max})$ formal definieren und alle Parameter erklären. → *Def. 2.13*

- ☐ $x_{\min} = \beta^{e_{\min}-1}$ und $x_{\max} = \beta^{e_{\max}}(1 - \beta^{-t})$ herleiten. → *Lemma 2.16*

- ☐ Erklären, warum Gleitkommazahlen auf dem Zahlenstrahl nicht gleichmäßig verteilt sind, aber gleichmäßige relative Genauigkeit haben. → *Lemma 2.17*

- ☐ Denormalisierte Zahlen erklären: Zweck (Lücke zu 0 schließen) und Nachteil (geringere relative Genauigkeit). → *§2.2*

- ☐ IEEE 754: Single vs. Double – Bit-Aufteilung (Vorzeichen/Exponent/Mantisse) und Zahlbereiche angeben. → *§2.3*

- ☐ “Implizites erstes Bit” bei IEEE 754 erklären. Warum gewinnt man dadurch 1 Bit Mantissenlänge? → *§2.3*

- ☐ Rundungseinheit $u = \frac{1}{2}\beta^{1-t}$ definieren; für Double und Single konkrete Werte nennen. → *Def. Rundungseinheit*

- ☐ Satz 2.20 formulieren: $\text{rd}(x) = x(1 + \delta)$, $|\delta| \leq u$; Beweisidee skizzieren. → *Satz 2.20*

- ☐ Maschinenoperationen $\oplus, \ominus, \otimes, \oslash$ definieren; begründen, warum Assoziativität nicht gilt. → *§2.4*

- ☐ Algorithmen A und B für $x_1^2 - x_2^2$ vergleichen; erklären, wann A unter Auslöschung leidet. → *Bsp. 2.24*

- ☐ Rekursion $I_k = e - kI_{k-1}$: Fehlerfortpflanzung $\Delta I_k \approx k! \Delta I_0$ erklären; warum ist dies katastrophal? → *Bsp. 2.25*

- ☐ Numerische Auslöschung definieren und am Beispiel $(1 - \cos x)/x^2$ zeigen; stabilen Alternativalgorithmus angeben. → *§2.5*
